CHƯƠNG 2

Ứng dụng AI tạo sinh trong phát triển phần mềm

[Ứng dụng Trí tuệ nhân tạo]



[Nguyễn Tuấn Anh]
• [Thái Nguyên, 2026]

Giới thiệu nội dung

Mục tiêu chương

- Phân tích được vai trò của AI tạo sinh trong từng giai đoạn của SDLC và mô tả được mô hình AI-Augmented SDLC với nguyên tắc Human-in-the-loop.
- Phân loại và lựa chọn được các công cụ AI hỗ trợ phát triển phần mềm (GitHub Copilot, Amazon Q Developer, ChatGPT, v.v.) phù hợp với ngữ cảnh dự án.

Nội dung chính

1. Vai trò của AI trong các pha SDLC
2. Các công cụ AI hỗ trợ phát triển
3. Đánh giá hiệu quả sử dụng AI
4. Thực hành với AI trong lập trình

Khởi động: Phát triển phần mềm đang thay đổi như thế nào?

- **Google**: Hơn **25%** code mới được tạo bởi AI (2024).
- **Amazon**: Dùng Amazon Q Developer di chuyển hàng chục ngàn ứng dụng Java 8/11 lên Java 17 – tiết kiệm **4.500 năm công** và **260 triệu USD/năm**.
- **Microsoft**: GitHub Copilot tăng tốc độ coding lên **30–40%**.
- **Meta**: CodeCompose giảm hơn **20%** công việc lập trình lặp lại.

Câu hỏi then chốt

AI có thể tham gia vào những khâu nào trong SDLC? Nó thay đổi vai trò của người phát triển ra sao? Đâu là giới hạn mà con người vẫn phải nắm quyền kiểm soát?

AI trong phân tích yêu cầu và lập kế hoạch

- **Phân loại và cấu trúc yêu cầu**: LLM đọc tài liệu yêu cầu thô (email, biên bản họp, văn bản tự do) và tự động phân loại thành yêu cầu chức năng / phi chức năng.
- **Phát hiện mâu thuẫn và thiếu sót**: Công cụ ALICE đạt độ chính xác **99%** và độ bao phủ 60% trong phát hiện mâu thuẫn giữa các yêu cầu. AI còn đánh giá được tính nhất quán, hoàn chỉnh và khả thi.
- **Hỗ trợ lập kế hoạch**: AI ước lượng công sức, đề xuất phân rã công việc (WBS), dự đoán rủi ro và sinh bản nháp kế hoạch dự án.

Lưu ý

AI không thể thay thế con người: hiểu sâu ngữ cảnh nghiệp vụ, đàm phán ưu tiên và ra quyết định trade-off vẫn đòi hỏi kinh nghiệm và phán đoán của con người.

AI trong thiết kế hệ thống

- **Sinh mô hình từ mô tả tự nhiên**: Mô tả hệ thống bằng tiếng Việt $\rightarrow$ AI tạo use case diagram, class diagram, sequence diagram. Hữu ích trong giai đoạn khám phá – nhanh chóng phác thảo và so sánh nhiều phương án.
- **Hỗ trợ thiết kế giao diện người dùng**: Figma AI, PromptInfuser – mô tả giao diện mong muốn bằng văn bản $\rightarrow$ nhận bản phác thảo UI trong vài phút.
- **Đề xuất và đánh giá kiến trúc**: AI như "kiến trúc sư đối thoại" – đề xuất phương án kiến trúc kèm phân tích ưu nhược điểm, trade-off và architectural patterns phù hợp.

Kết quả sinh bởi AI nên được xem như **bản nháp khởi đầu** để con người tinh chỉnh – AI hiện vẫn hạn chế về cấu trúc đầu ra và khả năng xử lý các mối quan hệ phức tạp.

AI trong lập trình và hiện thực

Đây là pha AI tạo ra tác động **rõ rệt nhất**. Theo DORA 2024: **75.9%** lập trình viên đã dùng AI trong công việc hàng ngày.

- **Sinh mã từ mô tả tự nhiên**: Copilot, Q Developer, ChatGPT tạo hàm/lớp/module từ mô tả. Giảm đến **40%** thời gian viết boilerplate code.
- **Hoàn thiện mã thông minh**: Phân tích toàn bộ ngữ cảnh dự án để gợi ý – 90% lập trình viên từng commit mã do Copilot gợi ý, 95% cảm thấy coding thú vị hơn.
- **Hỗ trợ gỡ lỗi**: Phân tích stack trace, xác định nguyên nhân gốc rễ, đề xuất bản vá – giảm **37%** thời gian giải quyết lỗi.
- **Tái cấu trúc và sinh tài liệu**: Đề xuất cải thiến cấu trúc, tự động sinh docstring và API reference.

AI trong kiểm thử và đảm bảo chất lượng

- **Sinh ca kiểm thử tự động**: AI đọc mã nguồn → sinh unit test bao phủ happy path, edge cases và error cases. Có nghiên cứu báo cáo **tăng 20% test coverage** chỉ trong một sprint.
- **Phát hiện lỗi và lỗ hổng bảo mật**: Tích hợp vào pipeline CI/CD, quét mã nguồn khi có PR mới – phát hiện SQL injection, hardcoded credentials, XSS. Công cụ như DeepCode (Snyk) phân tích theo thời gian thực.
- **Sinh script kiểm thử đầu cuối**: AI sinh script mô phỏng toàn bộ luồng người dùng với hàng trăm biến thể dữ liệu đầu vào.
- **AI trong code review**: Tự động phân tích tĩnh, phát hiện mẫu mã không hiệu quả, vi phạm chuẩn mã hóa – giảm tới **30%** thời gian review PR.

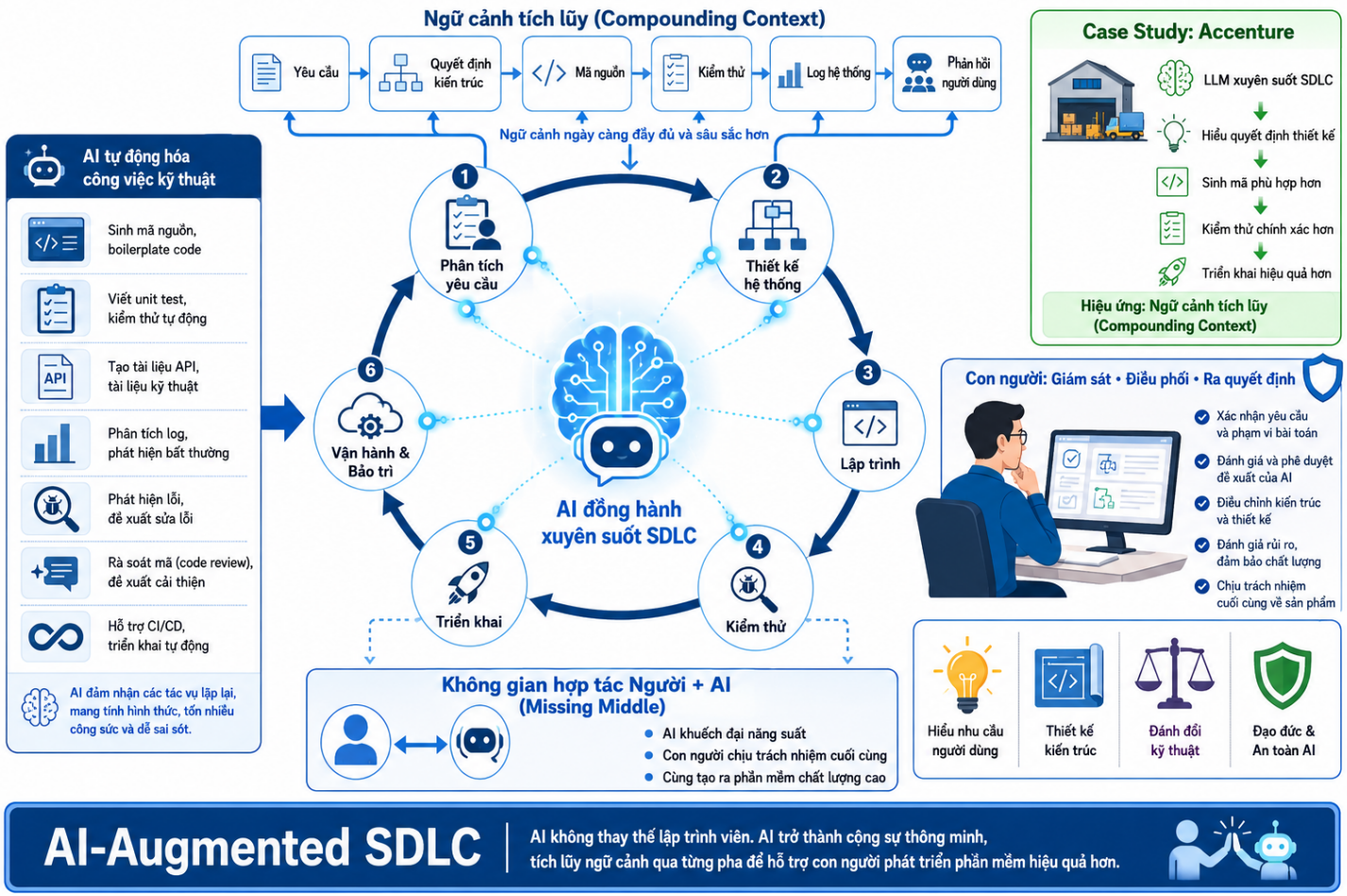
AI trong triển khai và vận hành

- **Sinh cấu hình triển khai**: AI tạo Docker, Kubernetes, GitHub Actions, Jenkins config từ mô tả tự nhiên.
- **Giám sát và phát hiện bất thường**: AI-powered observability – phân tích log, metrics và hành vi người dùng theo thời gian thực $\rightarrow$ chuyển từ "phản ứng" sang "chủ động ngăn chặn".
- **Hỗ trợ xử lý sự cố (RCA)**: Tổng hợp log, thay đổi mã gần đây, cấu hình môi trường $\rightarrow$ xác định nguyên nhân gốc rễ và đề xuất khắc phục.
- **Dự đoán năng lực**: Phân tích xu hướng sử dụng tài nguyên, dự đoán nhu cầu tương lai, đề xuất chiến lược mở rộng chủ động.

Lưu ý

35.8% Kubernetes manifest do ChatGPT tạo ra chứa ít nhất một "smell" – AI-generated infrastructure code cũng cần được kiểm tra chất lượng.

AI-Augmented SDLC: Bức tranh toàn cảnh



Compounding Context

Nguyên tắc Human-in-the-Loop (HITL)

AI xử lý tác vụ lặp lại; con người giám sát, xác nhận và chịu trách nhiệm.

Tại sao HITL là bắt buộc?

- **AI có thể tạo mã đúng cú pháp nhưng sai ngữ nghĩa** – chỉ con người với hiểu biết nghiệp vụ mới xác nhận được.
- **AI có thể "ảo giác" (hallucinate)** – gọi hàm API không tồn tại, đề xuất kiến trúc phi thực tế.
- **AI không chịu trách nhiệm** – khi hệ thống gặp sự cố, kỹ sư phần mềm chịu trách nhiệm, không phải mô hình AI.
- **AI tạo mã chứa lỗ hổng bảo mật** – khoảng 40% code Copilot chứa lỗ hổng trong thử nghiệm.

Phased Integration

Bắt đầu với tác vụ rủi ro thấp, phạm vi hẹp $\rightarrow$ mở rộng dần khi có đủ

Tổng kết Section 1 – Tự kiểm tra

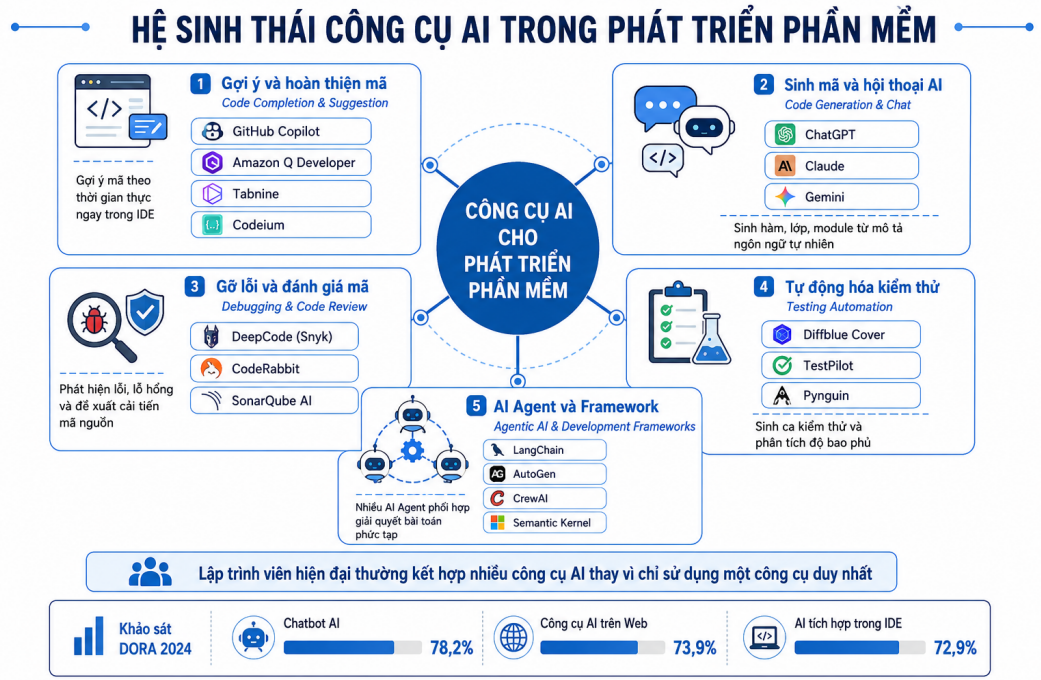
Những điểm cần nhớ

1. **AI hiện diện trong toàn bộ SDLC**: phân tích yêu cầu → thiết kế → lập trình → kiểm thử → triển khai và vận hành. Mỗi pha có cơ hội và thách thức riêng.
2. **AI-Augmented SDLC** với "compounding context" cho lợi ích vượt xa việc dùng AI rời rạc – ngữ cảnh tích lũy qua các pha giúp AI ngày càng phù hợp hơn.
3. **Human-in-the-Loop là nguyên tắc sống còn**: AI tăng tốc và hỗ trợ, con người giám sát, xác nhận và chịu trách nhiệm cuối cùng.

Tự kiểm tra nhanh:

1. Với mỗi pha SDLC, nêu một ví dụ AI hỗ trợ và một rủi ro cần lưu ý.
2. Giải thích "compounding context" và tại sao nó là lợi thế của AI-Augmented SDLC.

Bức tranh toàn cảnh: Phân loại công cụ AI



Năm nhóm chính:

1. Code Completion & Suggestion
2. Code Generation & Chat
3. Debugging & Code Review

DORA 2024

Chatbot (78.2%), công cụ web (73.9%), AI trong IDE (72.9%) – lập trình viên thường **kết hợp**

GitHub Copilot – Người lập trình cặp AI

- **Công cụ AI pair programmer phổ biến nhất toàn cầu**, dựa trên OpenAI Codex, tích hợp vào VS Code, JetBrains, Neovim.
- **Cách hoạt động**: Phân tích toàn bộ ngữ cảnh (file hiện tại, file liên quan, lịch sử chỉnh sửa) $\rightarrow$ gợi ý "ghost text" – chấp nhận bằng Tab.
- **Hiệu quả đã kiểm chứng**:
  - Hoàn thành tác vụ nhanh hơn **55.8%** (2.7h $\rightarrow$ 1.2h), tỷ lệ thành công 78% vs 70%.
  - 90% lập trình viên từng commit mã Copilot; 95% cảm thấy coding thú vị hơn.

Lưu ý

Tỷ lệ chấp nhận gợi ý chỉ **30%** – 70% bị từ chối. Khoảng **40%** code Copilot trong thử nghiệm chứa lỗ hổng bảo mật.

Amazon Q Developer – Trợ lý AI cho hệ sinh thái AWS

- **Điểm khác biệt với Copilot**:
  1. **Tích hợp sâu AWS**: Hiểu rõ SDK, API và best practices của DynamoDB, S3, Lambda.
  2. **Code security scanning**: Tích hợp sẵn trình quét bảo mật.
  3. **Code transformation**: Tự động nâng cấp phiên bản ngôn ngữ.
- **Hiệu quả**: Nhanh hơn 57%, khả năng thành công +27%.
- **Miễn phí cho cá nhân** (Copilot yêu cầu trả phí).

Case study nổi bật

Amazon di chuyển **hàng chục ngàn** ứng dụng Java 8/11 $\rightarrow$ Java 17:

- Tiết kiệm 4.500 năm công.
- Tiết kiệm 260 triệu USD/năm.

Khi nào chọn Q Developer?

Dự án chạy trên hạ tầng AWS và dùng các dịch vụ

ChatGPT và các nền tảng AI đa năng

- **Cách tiếp cận khác**: Không tích hợp sâu vào IDE – AI như "đối tác đối thoại" (conversational partner) trong suốt quá trình phát triển.
- **Thế mạnh**:
  - Giải thích mã nguồn phức tạp (top 3 tác vụ AI-supported theo DORA 2024).
  - Thiết kế kiến trúc và thảo luận trade-off.
  - Sinh tài liệu kỹ thuật, so sánh công nghệ, phân tích lỗi từ stack trace.
- **Chiến lược "tiered model utilization"**:
  - **GPT-4o / Claude**: giao tiếp tự nhiên, brainstorming, sinh mã vừa phải.
  - **O1 / O3**: suy luận phức tạp, thiết kế thuật toán, tối ưu hiệu năng.
  - **GPT-4.1 / Gemini 1.5 Pro**: xử lý ngữ cảnh cực lớn (1M+ token).

Copilot tỏa sáng ở "viết mã nhanh"; ChatGPT mạnh hơn ở "**suy nghĩ cùng lập trình viên**".

Các IDE tích hợp AI & Công cụ chuyên biệt

IDE với AI tích hợp – xu hướng AI trở thành tính năng mặc định:

- **VS Code**: Copilot, IntelliCode – hệ sinh thái extension phong phú nhất.
- **JetBrains**: Copilot, JetBrains AI Assistant, Tabnine – code analysis chuyên sâu.
- **Cursor**: AI-native, thiết kế từ đầu cho AI-assisted coding.
- **Windsurf**: AI flow paradigm, tự động phân tích toàn dự án.

Công cụ chuyên biệt:

- **Tabnine**: Hỗ trợ local model – phù hợp tổ chức yêu cầu cao về privacy.
- **DeepCode (Snyk)**: Phân tích mã theo thời gian thực, phát hiện lỗ hổng bảo mật.
- **Diffblue Cover**: Tự động sinh unit test cho Java.
- **LangChain / AutoGen / CrewAI**: Framework xây dựng hệ thống multi-agent.

Lựa chọn công cụ: Không có "tốt nhất" tuyệt đối

Năm yếu tố quyết định:

1. **Môi trường triển khai**: AWS → Q Developer; đa nền tảng → Copilot / ChatGPT.
2. **Ngôn ngữ lập trình**: Copilot tốt với hầu hết ngôn ngữ phổ biến; công cụ chuyên biệt có thể tốt hơn trong hệ sinh thái của chúng.
3. **Yêu cầu bảo mật**: Không gửi code lên cloud → Tabnine (local model); tuân thủ GDPR/HIPAA → kiểm tra chính sách dữ liệu.
4. **Chi phí**: Copilot $10/tháng; Q Developer, Codeium miễn phí cá nhân; ChatGPT Plus $20/tháng.
5. **Loại tác vụ**: Sinh mã nhanh → Copilot/Q Developer; thảo luận kiến trúc → ChatGPT/Claude; review mã → DeepCode; multi-agent → LangChain/AutoGen.

Xu hướng hội tụ

Ranh giới giữa các nhóm công cụ đang mờ dần – các công cụ đang hội tụ về một "**trợ lý phát triển toàn diện**" xuyên suốt toàn bộ SDLC

Tổng kết Section 2 – Tự kiểm tra

Những điểm cần nhớ

1. **GitHub Copilot** = pair programmer phổ biến nhất – mạnh về sinh mã trong IDE, cần thận trọng với bảo mật.
2. **Amazon Q Developer** = lựa chọn tối ưu cho AWS – miễn phí cá nhân, tích hợp security scanning.
3. **ChatGPT / Claude** = đối tác đối thoại – mạnh về suy nghĩ cùng lập trình viên hơn là viết mã thay lập trình viên.
4. **Không có công cụ "tốt nhất"** – lựa chọn phụ thuộc vào môi trường, ngôn ngữ, bảo mật, chi phí và loại tác vụ.

Tự kiểm tra nhanh:

1. Phân biệt điểm mạnh chính của Copilot, Q Developer và ChatGPT.
2. Dự án backend Python trên AWS, frontend React – bạn chọn công cụ AI nào? Tại sao?

AI có thực sự giúp chúng ta nhanh hơn?

"Nhanh hơn 55%", "tiết kiệm 4.500 năm công", "95% hài lòng hơn"…
Những con số này đến từ đâu? Trong điều kiện nào? Có khái quát hóa được không?

- Là kỹ sư phần mềm tương lai, bạn cần **đánh giá khách quan**, không chỉ tin vào quảng cáo.
- **Hiệu quả** trong phát triển phần mềm là khái niệm **đa chiều** – không chỉ "nhanh hơn".
- Cần **framework** để đo lường một cách có hệ thống.

Mục này trang bị cho bạn bộ công cụ để tự đánh giá – không chỉ cho AI, mà cho **mọi công nghệ và phương pháp mới** trong sự nghiệp.

DORA Metrics – Đo lường hiệu suất hệ thống

DORA (DevOps Research and Assessment) – tiêu chuẩn được chấp nhận rộng rãi nhất để đo lường hiệu suất phân phối phần mềm:

1. **Deployment Frequency** (Tần suất triển khai): Elite performers triển khai nhiều lần/ngày.
2. **Lead Time for Changes** (Thời gian commit $\rightarrow$ production): Elite performers dưới 1 giờ.
3. **Change Failure Rate** (Tỷ lệ triển khai thất bại): Elite performers duy trì 0–15%.
4. **Mean Time to Recovery** (Thời gian khôi phục trung bình): Elite performers dưới 1 giờ.

DORA và AI

AI-powered code review $\rightarrow$ giảm change failure rate. AI-generated CI/CD scripts $\rightarrow$ tăng deployment frequency. AI-assisted incident analysis $\rightarrow$ giảm MTTR. Nhưng DORA đo ở cấp **hệ thống/đội nhóm**, không đo trải nghiệm từng cá nhân.

SPACE Framework – Mở rộng ra trải nghiệm con người

SPACE (GitHub, Microsoft, University of Victoria) bổ sung chiều con người:

- **S**atisfaction & Well-being: 87% lập trình viên tiết kiệm năng lượng tinh thần nhờ Copilot; 95% thấy coding thú vị hơn.
- **P**erformance: Tần suất triển khai, thời gian hoàn thành tác vụ, số tính năng phân phối.
- **A**ctivity: Số commit, PR, code review, test case – **activity** $\neq$ **value**.
- **C**ommunication & Collaboration: Chất lượng phối hợp nhóm, thời gian chờ review.
- **E**fficiency & Flow: Khả năng tập trung không gián đoạn – AI code completion giảm context switching.

Lưu ý

Activity tăng không đồng nghĩa với value tăng – nhiều commit hơn nhờ AI nhưng chất lượng mỗi commit có thể thấp hơn.

DevX – Trải nghiệm lập trình viên

DevX (Developer Experience) tập trung vào tương tác giữa lập trình viên, quy trình và công cụ:

- **Thời gian mất do công cụ** (tool delays): chậm, lỗi, gián đoạn.
- **Thời gian onboarding**: AI giảm thời gian làm quen dự án qua giải thích mã và sinh tài liệu tự động.
- **Tải nhận thức** (cognitive overhead): AI giảm context switching (trả lời ngay trong IDE).
- **Cảm nhận chủ quan** (developer sentiment).

AI tác động đến DevX theo cả hai hướng:

- Tích cực: giảm context switching, giảm thời gian onboarding.
- Tiêu cực: sửa gợi ý sai của AI $\rightarrow$ giảm DevX; ép dùng AI không đào tạo $\rightarrow$ sentiment giảm.

Bằng chứng định lượng: Tổng hợp nghiên cứu

Nghiên cứu	Công cụ	Kết quả chính	Ghi chú
GitHub Copilot (thử nghiệm)	Copilot	Nhanh hơn 55.8%; thành công 78% vs 70%	Có kiểm soát
Amazon Q Developer	Q Developer	Nhanh hơn 57%; thành công +27%	Tối ưu AWS
AI-powered code review	Copilot, CodeRabbit	Giảm 30% thời gian review PR	Tích hợp CI/CD
AI-generated testing	ChatGPT, Pynguin	Tăng 20% test coverage/sprint	Cần human review
Boilerplate generation	Copilot, ChatGPT	Giảm 96% thời gian viết boilerplate	Pattern rõ ràng
Amazon Java migration	Q Developer	4.500 năm công, $260M/năm	Java 8/11 $\rightarrow$ 17

Lưu ý

Hầu hết nghiên cứu trong môi trường có kiểm soát hoặc do chính nhà cung cấp thực hiện. Hiệu quả thực tế phụ thuộc **ngữ cảnh**: loại tác vụ, kinh nghiệm lập trình viên, chất lượng prompt.

Mặt trái: Khi AI không giúp chúng ta nhanh hơn

- **Nghiên cứu METR (Becker et al. 2025)**: Thử nghiệm ngẫu nhiên có kiểm soát – lập trình viên dùng AI **chậm hơn 19%** so với không dùng AI.
- **Code churn tăng**: Nghiên cứu GitClear – lập trình viên chấp nhận gợi ý AI, sau đó phải sửa/viết lại $\rightarrow$ lãng phí công sức.
- **Rủi ro bảo mật**: **40%** code Copilot chứa lỗ hổng; **35.8%** Kubernetes manifest do ChatGPT tạo có "smell".
- **Over-reliance**: Nguy cơ mất khả năng tự giải quyết vấn đề – đặc biệt nguy hiểm với sinh viên và lập trình viên mới.

Bài học

AI không phải "cây đũa thần". Trong một số ngữ cảnh, nó có thể **làm giảm** hiệu quả. Cần đánh giá khách quan dựa trên **bằng chứng**, không dựa trên **cảm tính** hay **quảng cáo**.

Các trade-off chính khi tích hợp AI

1. **Chi phí hạ tầng và vận hành**: Fine-tuning thường xuyên $\rightarrow$ áp lực tài nguyên. Chiến lược: lập lịch fine-tuning theo major release; dùng SLM cho tác vụ đơn giản.
2. **Kiểm soát và sở hữu mã nguồn**: AI làm suy yếu cảm giác "ownership". Giải pháp: chế độ suggestion-only; lập trình viên phải **hiểu** mã trước khi chấp nhận.
3. **Bảo mật và tuân thủ**: Gửi code lên cloud có thể vi phạm GDPR, HIPAA. Giải pháp: data anonymization, local model (Tabnine), chính sách rõ ràng.
4. **Chất lượng và nợ kỹ thuật**: Code churn tăng, test coverage không đầy đủ. Giải pháp: "phased integration" – bắt đầu rủi ro thấp, đo lường, mở rộng dần.
5. **Niềm tin của lập trình viên**: Một lỗi nghiêm trọng do AI $\rightarrow$ từ chối mọi gợi ý sau đó. Xây dựng niềm tin cần **thời gian và minh bạch**.

Tổng kết Section 3 – Tự kiểm tra

Những điểm cần nhớ

1. **DORA, SPACE, DevX** cung cấp các lăng kính đa chiều để đo lường – không chỉ "nhanh hơn" mà còn "tốt hơn", "khỏe hơn" và "bền vững hơn".
2. **Bằng chứng định lượng** cho thấy AI cải thiện tốc độ 30–55%, giảm 30% thời gian review, tăng 20% test coverage – nhưng phụ thuộc nhiều vào ngữ cảnh.
3. **Mặt trái là có thực**: code churn tăng, 40% code AI chứa lỗ hổng bảo mật, nguy cơ over-reliance, có nghiên cứu cho thấy AI làm chậm hơn 19%.
4. **Tích hợp thông minh**: Cân nhắc trade-off về chi phí, kiểm soát, bảo mật, chất lượng và niềm tin. Phased integration $\rightarrow$ kết quả bền vững.

Kỹ thuật thiết kế prompt cho lập trình

Ba kỹ thuật nền tảng – "bộ công cụ tư duy" quyết định chất lượng đầu ra:

1. **Zero-Shot Prompting**: Yêu cầu trực tiếp, không kèm ví dụ.
   - Phù hợp: tác vụ đơn giản, pattern rõ ràng, có nhiều ví dụ trong dữ liệu huấn luyện.
   - Hạn chế: kết quả không nhất quán với tác vụ phức tạp.
2. **Few-Shot Prompting**: Cung cấp vài ví dụ mẫu trước yêu cầu thực sự.
   - Thiết lập pattern về định dạng, phong cách code, mức xử lý lỗi, cách đặt tên.
   - Đặc biệt hiệu quả khi muốn AI tuân theo quy ước code nhất quán.
3. **Chain-of-Thought (CoT)**: Yêu cầu AI "suy nghĩ thành tiếng" từng bước.
   - Giảm thiểu bỏ sót edge case, chọn sai thuật toán.
   - Đặc biệt hiệu quả với bài toán logic, giải thuật và gỡ lỗi phức tạp.

Sinh mã nguồn với AI

- **Sinh hàm đơn lẻ**: Mô tả bằng ngôn ngữ tự nhiên kèm signature hàm – AI đặc biệt hiệu quả với xử lý chuỗi, mảng, CRUD, chuyển đổi định dạng dữ liệu.
- **Sinh module và cấu trúc dự án**: "Phân rã rồi chinh phục" – chia yêu cầu lớn thành các bước, yêu cầu AI đề xuất cấu trúc thư mục trước, sinh từng file sau.

Ví dụ prompt sinh mã

```
Viết hàm Python tên là is_valid_email kiểm tra xem một
chuỗi có phải là địa chỉ email hợp lệ hay không.
Sử dụng regex. Có type hints và docstring tiếng Việt.
```

Code review sau khi sinh – không thể bỏ qua

Mã có chạy được không? Logic có đúng không? Có lỗ hổng bảo mật

Gỡ lỗi với AI

Pattern hiệu quả nhất: cung cấp **code + input + stack trace**:

Prompt debug mẫu

```
Tôi đang gặp lỗi với đoạn code Python sau:
[PASTE CODE]
Khi chạy với đầu vào [MÔ TẢ], tôi nhận được lỗi:
[PASTE STACK TRACE]
Hãy:
1. Phân tích nguyên nhân gốc rễ
2. Đề xuất cách sửa cụ thể (kèm code)
3. Giải thích tại sao lỗi xảy ra và cách phòng tránh
```

- AI phân tích lượng lớn thông tin (stack trace, log, mã nguồn) nhanh hơn con người rất nhiều.
- Với lỗi phức tạp (race condition, memory leak, logic nghiệp vụ đặc thù): AI nên là “bộ lọc sơ bộ”, không phải “người sửa lỗi cuối cùng”.

Sinh kiểm thử với AI

- AI đọc một hàm và tự động đề xuất unit test bao phủ **happy path, edge cases, error cases**.
- AI đặc biệt giỏi nghĩ ra edge case mà lập trình viên thường bỏ qua: chuỗi rỗng, số âm, None, danh sách rỗng, ký tự Unicode, injection attacks.

Prompt sinh test

```
Cho hàm Python sau: [PASTE CODE]
Hãy viết unit test bằng pytest, bao phủ:
1. Happy path
2. Edge cases (biên, rỗng, giá trị đặc biệt)
3. Error cases (đầu vào không hợp lệ)
4. Performance edge case
```

Lưu ý

Sinh tài liệu kỹ thuật với AI

- Một trong những ứng dụng AI có **ROI cao nhất**: giảm thời gian cho công việc lập trình viên thường trì hoãn, cải thiện chất lượng tài liệu toàn đội.

Hai pattern chính:

1. **Sinh docstring và chú thích**:

```
Viết docstring cho hàm Python sau theo chuẩn Google style.
Bao gồm: mô tả, Args, Returns, Raises, Example.
[PASTE CODE]
```

2. **Sinh README và tài liệu API**:

```
Dưới đây là code của một module Python. Hãy viết tài liệu
API dạng Markdown: tổng quan, danh sách hàm public, hướng
dẫn cài đặt. [PASTE CODE]
```

Nguyên tắc

AI viết bản nháp, con người biên tập và xác nhận. Luôn đọc lại và

Quy trình quản lý prompt hiệu quả

Prompt là **tài sản** (asset) của dự án, cần quản lý có hệ thống:

1. **Prompt Design** – Thiết kế prompt ban đầu: mục tiêu, ngữ cảnh, định dạng đầu ra, ràng buộc.
2. **Testing & Evaluation** – Kiểm thử với nhiều đầu vào, đánh giá có hệ thống.
3. **Refinement** – Tinh chỉnh dựa trên kết quả: thay đổi cách diễn đạt, thêm ví dụ, điều chỉnh ràng buộc.
4. **Versioning & Cataloging** – Lưu prompt đã kiểm chứng vào repo (thư mục `prompts/`), kèm ghi chú phiên bản và ngữ cảnh.
5. **Deployment** – Triển khai vào workflow: CI/CD, chia sẻ đội, nhúng vào công cụ nội bộ.
6. **Monitoring & Iteration** – Theo dõi chất lượng theo thời gian; cập nhật khi model nâng cấp hoặc yêu cầu thay đổi.

Ngay cả khi làm việc một mình, lưu prompt tốt vào file Markdown/text trong repo giúp tiết kiệm thời gian đáng kể.

Tổng kết thực hành & Kết nối

Qua 2 tiết thực hành, bạn đã làm việc với AI trong **bốn tác vụ lập trình cốt lõi**:

1. **Sinh mã nguồn**: Ba kỹ thuật prompt (Zero-Shot, Few-Shot, CoT) cho phép điều chỉnh chất lượng và phong cách code. Chọn kỹ thuật phù hợp với độ phức tạp của tác vụ.
2. **Gỡ lỗi**: Pattern "code + input + stack trace" là công thức hiệu quả nhất. AI giúp thu hẹp phạm vi tìm kiếm, nhưng kỹ năng debug của bạn vẫn là nền tảng.
3. **Sinh kiểm thử**: AI vượt trội trong việc nghĩ ra edge case – nhưng không hiểu ngữ cảnh nghiệp vụ. Luôn bổ sung test dựa trên hiểu biết miền ứng dụng.
4. **Sinh tài liệu**: AI viết bản nháp nhanh, con người biên tập và xác nhận tính chính xác.

Điểm mấu chốt

Prompt engineering là **kỹ năng kỹ thuật** (engineering skill), không

Bài tập về nhà

1. **So sánh kỹ thuật prompt**: Chọn một hàm Python bạn đã viết. Thiết kế ba prompt (Zero-Shot, Few-Shot, CoT) để yêu cầu AI sinh hàm đó. So sánh chất lượng code, khả năng chạy đúng, edge case được xử lý. Viết nhận xét ngắn (200 từ).
2. **Gỡ lỗi với AI**: Chạy file `debugging_with_ai.py` trong `content/examples/chapter02/section04/`. Chọn hai hàm có lỗi, dán prompt debug vào ChatGPT/Claude. Ghi lại phân tích của AI, tự sửa code, đánh giá độ chính xác.
3. **Sinh kiểm thử**: Chọn một hàm từ dự án cá nhân (ít nhất 10 dòng, 2 nhánh điều kiện). Yêu cầu AI sinh unit test. Liệt kê edge case AI tìm ra mà bạn chưa nghĩ tới, và trường hợp AI bỏ sót.
4. **Xây dựng bộ prompt cá nhân**: Tạo thư mục `prompts/` trong repo đồ án. Lưu ít nhất 3 prompt đã kiểm chứng cho sinh mã, gỡ lỗi và sinh kiểm thử.

Tổng kết chương

Những điểm cần ghi nhớ

- **AI-Augmented SDLC**: AI tham gia xuyên suốt các pha SDLC với hiệu ứng "compounding context". Vai trò lập trình viên chuyển từ creator sang validator/orchestrator.
- **Năm nhóm công cụ AI**: Code Completion, Code Generation & Chat, Debugging & Review, Testing Automation, Agentic AI Frameworks. Không có công cụ "tốt nhất" tuyệt đối – lựa chọn theo ngữ cảnh.
- **Đánh giá khách quan** qua DORA, SPACE, DevX. AI cải thiện tốc độ 30–55% nhưng cũng có mặt trái: 40% code Copilot chứa lỗ hổng bảo mật, code churn tăng, nguy cơ over-reliance.
- **Prompt engineering** với ba kỹ thuật nền tảng (Zero-Shot, Few-Shot, CoT). Quản lý prompt theo chu trình: Thiết kế $\rightarrow$ Kiểm thử $\rightarrow$ Tinh chỉnh $\rightarrow$ Versioning $\rightarrow$ Triển khai $\rightarrow$ Giám sát.
- **Human-in-the-Loop là bắt buộc**: AI là công cụ tăng cường